

From Notebooks to Production

Your notebooks work. Now turn them into a real inference service — scripts, tests, API, containers, CI/CD.

Before you start

- This task is bigger than the ones before it, and most of it you will learn on your own — that is the point
- Main reference: madewithml.com — read it, follow the structure, understand the reasoning behind it
- Search, read the official docs of every tool, ask ChatGPT or any AI assistant, ask in the classroom
- Do not copy a repo you cannot explain. Build it piece by piece and know why each piece is there

Where we are now

- ✓ Six notebooks that run in order and save their artifacts
- ✓ A trained model, fitted transformers, and a feature list saved from Notebook 5 and 6
- ✓ You can explain your split, your features, and your metric

What we build now

- The inference pipeline only — training stays in the notebooks for this task
- Input: a new order. Output: late or on time, with a probability
- The same fitted objects from your notebooks are loaded, never re-fitted
- Everything runs from the command line and from an API, inside containers

The work

1

Repository & configuration

- **Set up a clean folder structure: app, config, data, models, notebooks, src, tests, requirements**
- **No hardcoded paths, no hardcoded parameters — everything comes from a config file**
- Pin your dependencies, and separate runtime requirements from development requirements
- Add a README that explains how to run the project from zero

2

Notebooks to Python modules

- **Refactor each notebook into functions and modules, not one long script**
- Separate the concerns: data access, validation, preprocessing, features, prediction
- **Load the saved fitted objects — encoders, scalers, imputers — never fit again at inference**
- **The pipeline must reproduce exactly what your notebook produced on the same input**

3

Logging & error handling

- **Use the logging library, not print statements**
- Log levels, log format, log to file and to console
- Log every prediction request: input, output, latency, model version
- Handle bad input and missing values without crashing the service

4

Data versioning & validation

- **Version your data and artifacts with DVC, so any result can be traced back**
- **Validate incoming data with Great Expectations before it reaches the model**
- Define your expectations: column types, ranges, allowed categories, missing rates
- Decide what the service does when validation fails — reject, flag, or default

5

Experiment tracking & model registry

- **Track runs, parameters, metrics, and artifacts with MLflow**
- Register the model you chose, with a version and a stage
- **The service loads the model from the registry or the artifact store, not from a local notebook folder**
- Store artifacts somewhere the container can reach — not on your laptop only

6

Testing

- **Unit tests for your functions: preprocessing, feature building, utilities**
- Data tests: schema, ranges, nulls, leakage checks
- Model tests: the model loads, predicts the right shape, behaves on known inputs
- **Integration tests for the API routes end to end**
- Use pytest, and make the tests runnable with one command

7

The API

- **Build the service with FastAPI**
- **Routes: health check, model info and version, predict for a single order, batch predict**
- Validate the request and the response with schemas — reject bad payloads clearly
- Return the prediction, the probability, and the model version
- Check the automatic API docs and make sure the examples work

8

Docker & Docker Compose

- **Write a Dockerfile for the service — small image, no notebooks inside**
- **Compose the stack: the database, the API, and storage for the artifacts**
- **The whole system must come up with one command on a clean machine**
- Use environment variables for secrets and connection strings, never commit them

9

CI/CD

- **A pipeline that runs on every push: lint, format check, tests**
- Build the image and push it if everything passes
- Add pre-commit hooks so problems are caught before the push
- **A failing test must stop the pipeline — that is the whole point**

10

Monitoring

- **Expose service metrics: request count, latency, error rate**
- **Track prediction distribution over time, and watch for drift**
- **Store prediction logs so you can evaluate later when the real delivery date arrives**
- Decide what you would alert on, and write it down

Definition of done — you should have

- ✓ 1. A structured repository with config files, requirements, and a README that works from zero
- ✓ 2. An inference pipeline in Python modules that loads the saved fitted objects and the registered model
- ✓ 3. Data versioning with DVC, data validation with Great Expectations, tracking with MLflow
- ✓ 4. Unit tests and integration tests that pass with one command
- ✓ 5. A FastAPI service with health, model info, and predict routes
- ✓ 6. Docker Compose bringing up the database, the API, and the artifact storage together
- ✓ 7. A CI/CD pipeline that runs the tests and builds the image
- ✓ 8. Logging and monitoring in place, with prediction logs stored

Definition of done — you should be able to

- ✓ 1. Clone the repo on a clean machine and start everything with one command
- ✓ 2. Send an order to the predict route and get back a prediction, a probability, and a model version
- ✓ 3. Show that the pipeline output matches your notebook output on the same input
- ✓ 4. Break something on purpose — bad data, a failing test — and show the system catches it
- ✓ 5. Explain every folder, every config value, and every tool you used, and why it is there

Notes

- Do it in order — the API is easier once the modules and tests exist
- Small commits, meaningful messages, and keep the repo clean
- No training inside the inference pipeline — that comes in a later task
- If a tool is new to you, spend time on its documentation before you write code

Good luck guys!

من النوتبوكس إلى الإنتاج

النوتبوكس تبعتك شغالة. هلاً نحولها لخدمة inference حقيقية — سكرينات، تيسات، API، كونتينرز، و CI/CD.

قبل ما تبلىش

- هذا التاسك أكبر من اللي قبله، ومعظمه رح تتعلمه بنفسك — وهذا هو المقصود
- المرجع الأساسي: madewithml.com — اقرأه، اتبع الهيكلية، وافهم السبب وراءها
- دور، اقرأ الدوكيومنتيشن الرسمي لكل أداة، اسأل ChatGPT أو أي مساعد ذكاء اصطناعي، أو اسأل بالكلاس
- لا تنسخ ريبو ما بتقدر تشرحه. ابنه قطعة قطعة واعرف ليش كل قطعة موجودة

وين صرنا هلاً

- ✓ ست نوتبوكس بيشغلوا بالترتيب ويحفظوا ال artifacts تبعهم
- ✓ موديل مدرب، وتحويالات مدربة، وقائمة فيتشرز محفوظة من نوتبوك 5 و 6
- ✓ تقدر تشرح التقسيم والفيتشرز وال metric اللي اخترتها

شو رح نبني هلاً

- بايلاين ال inference بس — التدريب بضل بالنوتبوكس بهذا التاسك
- المدخل: طلب جديد. المخرج: متأخر أو بالوقت، مع احتمالية
- نفس ال objects المحفوظة من نوتبوكسك بتنحمل، وما بتتدرب من جديد أبدا
- كل إشي بيشغل من ال command line ومن API، وجوا كونتينرز

الشغل

الريبو والإعدادات

1

- جهز هيكلية مجلدات نظيفة: app و config و data و models و notebooks و src و tests و requirements
- ما في مسارات مكتوبة بالكود، ولا قيم ثابتة — كل إشي بييجي من ملف config
- ثبت إصدارات المكتبات، وافصل متطلبات التشغيل عن متطلبات التطوير
- ضيف README يشرح كيف تشغل المشروع من الصفر

من النوتبوكس إلى موديولات بايثون

2

- حول كل نوتبوك لدوال وموديولات، مو سكرت واحد طويل
- افصل المسؤوليات: قراءة البيانات، التحقق، المعالجة، الفيتشرز، والتوقع
- حمل ال objects المحفوظة — encoders و scalers و imputers — وما تدربها من جديد وقت ال inference
- لازم البايلاين ينتج بالضبط نفس اللي طلع من النوتبوك على نفس المدخل

ال logging ومعالجة الأخطاء

3

- استعمل مكتبة logging، مو print
- مستويات ال log، وصيغته، وتسجيله بملف وعالشاشة
- سجل كل طلب توقع: المدخل، المخرج، الزمن، وإصدار الموديل
- تعامل مع المدخلات الخرابانة والقيم الناقصة بدون ما تقع الخدمة

إصدارات البيانات والتحقق منها

4

- عمل **versioning** للبيانات وال **artifacts** باستعمال **DVC** ، حتى أي نتيجة تقدر ترجعها لمصدرها
- تحقق من البيانات الداخلة باستعمال **Great Expectations** قبل ما توصل للموديل
- حدد توقعاتك: أنواع الأعمدة، المدايات، الفئات المسموحة، ونسب النقص
- قرر شو بتعمل الخدمة لما يفشل التحقق — ترفض، تعلم، أو تستعمل قيمة افتراضية

تتبع التجارب وسجل الموديلات

5

- تتبع ال **runs** والبارامترات وال **metrics** وال **artifacts** باستعمال **MLflow**
- سجل الموديل اللي اخترته، مع إصدار ومرحلة
- الخدمة بتحمل الموديل من ال **registry** أو مخزن ال **artifacts** ، مو من مجلد نوتبوك محلي
- خزن ال **artifacts** بمكان الكونتينر بيقدّر يوصله — مو بس على لابتوبك

الاختبارات

6

- لدوالك: المعالجة، بناء الفيتشرز، والأدوات المساعدة **unit tests**
- اختبارات البيانات: ال **schema** ، المدايات، القيم الفاضية، وفحص التسريب
- اختبارات الموديل: الموديل بيتحمل، بيتوقع بالشكل الصحيح، ويتصرف منيح على مدخلات معروفة
- من البداية للنهاية **API** لمسارات ال **integration tests**
- استعمال **pytest** ، وخلي التيسات تشتغل بأمر واحد

ال API

7

- ابن الخدمة باستعمال **FastAPI**
- المسارات: **health check** ، معلومات وإصدار الموديل، توقع لطلب واحد، وتوقع لدفعة
- تحقق من الطلب والرد باستعمال **schemas** — ارفض المدخلات الخرابانة بوضوح
- رجع التوقع والاحتمالية وإصدار الموديل
- افحص دوكيومنتيشن ال **API** التلقائي وتأكد إن الأمثلة شغالة

Docker و Docker Compose

8

- اكتب **Dockerfile** للخدمة — **image** صغير، وبدون نوتبوكس جواته
- جمع الستاك: قاعدة البيانات، وال **API** ، والتخزين لل **artifacts**
- لازم النظام كله يشتغل بأمر واحد على جهاز نظيف
- استعمال **environment variables** للأسرار وسلاسل الاتصال، وما ترفعهم عاليو أبدا

ال CI/CD

9

- بايبلين يشتغل مع كل **push : lint** وفحص التنسيق والتيسات
- ابن ال **image** وارفعه إذا كل إشي نجح
- ضيف **pre-commit hooks** حتى المشاكل تتمسك قبل ال **push**
- تيس فاشل لازم يوقف البايلين — وهذا كل الهدف

- اعرض metrics للخدمة: عدد الطلبات، الزمن، ونسبة الأخطاء
- تتبع توزيع التوقعات مع الوقت، وراقب ال drift
- خزن سجلات التوقعات حتى تقدر تقييمها لاحقاً لما يوصل تاريخ التوصيل الحقيقي
- قرر شو بدك تعمل عليه تنبيه، واكتبه

تعريف الإنجاز — لازم يكون عندك

- ✓ 1. ريبو منظم مع ملفات config ومتطلبات و README يشتغل من الصفر
- ✓ 2. بايبلين inference بموديولات بايثون يحمل ال objects المحفوظة والموديل المسجل
- ✓ 3. MLflow وتتبع بـ ، Great Expectations وتحقق بـ ، DVC للبيانات بـ versioning
- ✓ 4. بنتجج بأمر واحد integration tests و unit tests
- ✓ 5. خدمة FastAPI فيها مسارات health ومعلومات الموديل والتوقع
- ✓ 6. مع بعض artifacts وتخزين ال API يشتغل قاعدة البيانات وال Docker Compose
- ✓ 7. بايبلين CI/CD يشتغل التيسات ويبني ال image
- ✓ 8. ومراقبة شغالين، وسجلات التوقعات محفوظة logging

تعريف الإنجاز — لازم تقدر

- ✓ 1. تنسخ الريبو على جهاز نظيف وتشغل كل إشي بأمر واحد
- ✓ 2. تبعت طلب لمسار التوقع وترجعك نتيجة واحتمالية وإصدار الموديل
- ✓ 3. تثبت إن مخرج البايلين مطابق لمخرج النوتبوك على نفس المدخل
- ✓ 4. تخرب إشي عن قصد — بيانات خرابنة أو تيست فاشل — وتوري إن النظام مسكها
- ✓ 5. تشرح كل مجلد وكل قيمة config وكل أداة استعملتها، وليش موجودة

ملاحظات

- اشتغل بالترتيب — ال API يبصير أسهل لما الموديولات والتيسات تكون جاهزة
- صغيرة، برسائل واضحة، وخلي الريبو نظيف commits
- ما في تدريب جوا بايبلين ال inference — رح ييجي بتاسك لاحق
- إذا في أداة جديدة عليك، خذ وقتك بدوكيومنتيشن تبعثها قبل ما تكتب كود

بالتوفيق للجميع!